CN 2026-03-05

Interpolazione polinomiale.

Dato $f:[a,b]\to\mathbb{R}$, $a\le x_0<x_1<\dots<x_n\le b$

Allora $\exists\, p(x)\in\Pi_n$: $p(x_i)=f(x_i)\equiv f_i$, $i=0,\dots,n$.

- forma di Lagrange di $p(x)$
- forma di Newton di $p(x)$:

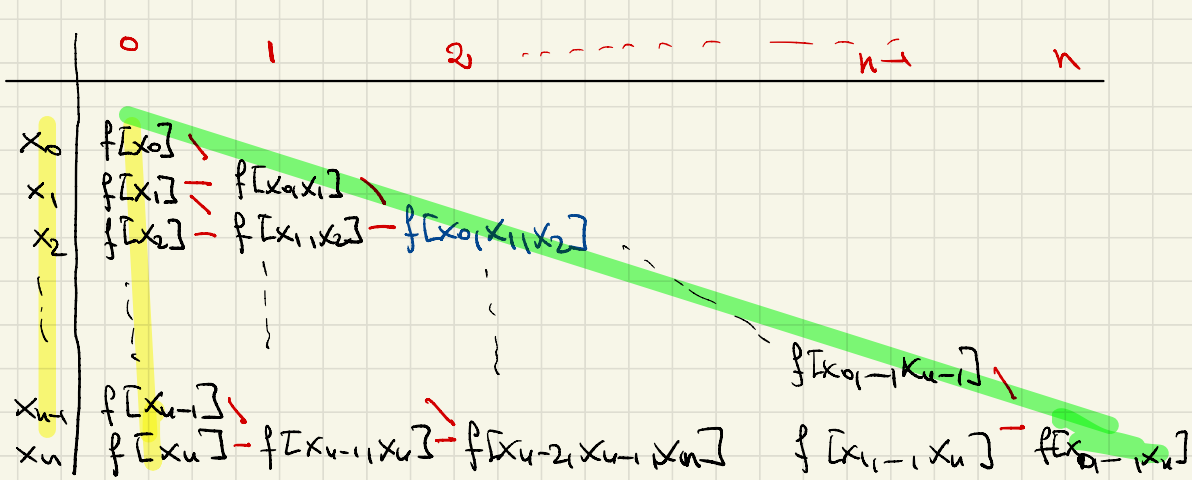
$$(1)\qquad p(x)=\sum_{i=0}^{n} f[\underbrace{x_0,\dots,x_i}_{i+1}]\,w_i(x),$$

$$(2)\qquad \begin{cases} w_i(x)=w_{i-1}(x)\cdot(x-x_{i-1}), & i\ge 1,\\ w_0(x)\equiv 1 \end{cases}$$

i cui coefficienti sono le **differenze divise** di $f(x)$ sulle ascisse in argomento.

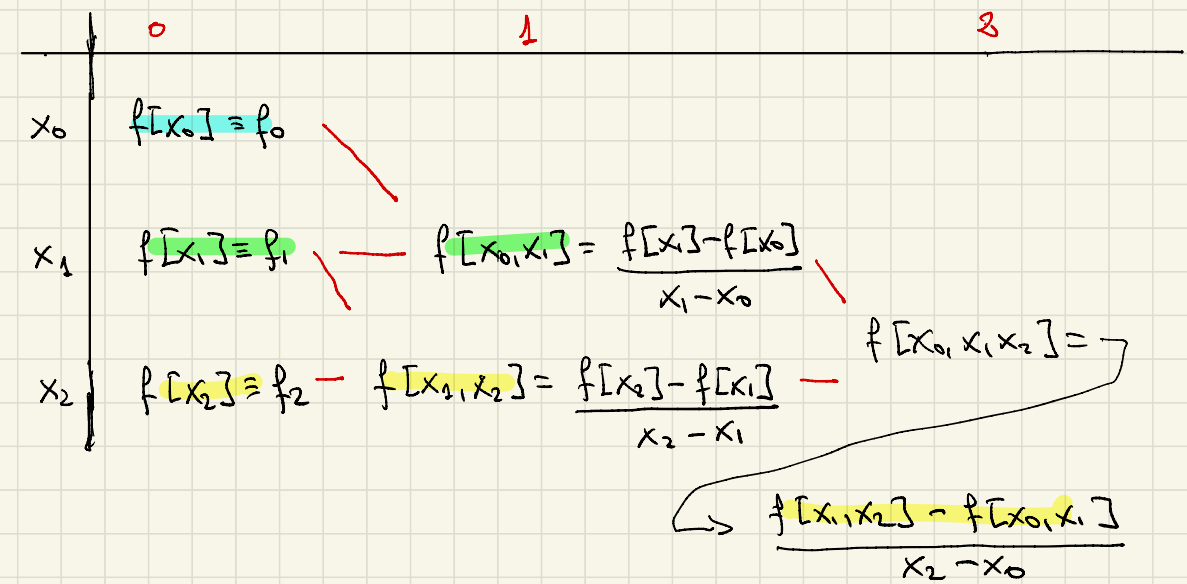
Utilizzando la proprietà "incrementale" per il calcolo delle differenze divise:

$$(3)\qquad \begin{cases} f[x_r]=f(x_r)=f_r \\ f[\underbrace{x_0,\dots,x_r}_{r+1}]=\dfrac{f[\overbrace{x_1,\dots,x_r}^{r}]-f[\overbrace{x_0,\dots,x_{r-1}}^{r}]}{x_r-x_0} \end{cases}$$



Esaminiamo, in dettaglio, il caso $n=2$, prima di derivare una procedura generale per il calcolo delle differenze divise nel caso di n generico.

$n=2$



Scriviamo, ora, uno pseudo-codice Matlab che implementa questo algoritmo nel caso generale. Poiché i vettori hanno indicizzazione a partire da 1, i vettori in ingresso saranno: x, f di lunghezza n+1. Pertanto

```
for j = 1:n
    for i = n+1:-1:j+1
        f(i) = ( f(i) - f(i-1) ) / ( x(i) - x(i-j) );
    end
end
```

(A)

oss questo pseudo-codice è, evidentemente, funzionante in modo corretto se e solo se il vettore x contiene elementi tra loro distinti. Questo controllo, a questo livello, va effettuato prima di eseguire le precedenti istruzioni.

Vediamo qual è il costo computazionale dell'algoritmo (A).

Occupazione di memoria: 2 vettori di lunghezza $n+1$ (x e f). Pertanto, la complessità è lineare in n.

Numero di operazioni: abbiamo 3 operazioni elementari nel ciclo più interno e, pertanto, otteniamo:

$$3 \sum_{j=1}^{n} (n-j+1) = 3 \sum_{j=1}^{n} j = 3 \frac{n(n+1)}{2} = O(n^2)$$

OSS:

$$\sum_{j=0}^{n} j^k \approx \int_0^n x^k \, dx = \frac{n^{k+1}}{k+1}$$

Quindi, complessità quadratica nel grado del polinomio interpolante.

Avendo compreso come calcolare in modo efficiente le differenze divise in (1), vediamo come calcolare efficientemente il polinomio $p(x)$.

Per ottenere la soluzione di questo problema, consideriamo, preliminarmente, un problema più semplice, ovvero il calcolo di un polinomio di grado n rispetto alla base delle potenze:

$$p(x) = \sum_{i=0}^{n} a_i x^i \qquad (5)$$

es: $n = 2$

$$p(x) = a_0 + a_1 x + a_2 x^2$$

	p
$p = a_2$	a_2
$p \leftarrow px + a_1$	$a_2 x + a_1$
$p \leftarrow px + a_0$	$(a_2 x + a_1)x + a_0 = \underbrace{a_2 x^2 + a_1 x + a_0}_{p(x)}$

In generale, per il polinomio in (5) avremo che:

```
p = a_n
per i = 1 : n
    p = p · x + a_{n-i}
fine per
```

$\Rightarrow$ p conterrà, alla fine, il valore di p(x)

Questo è l'algoritmo di Horner per il calcolo di un polinomio. La sua complessità è $2n$ flops.

Pseudo-codice Matlab (al solito, i vettori hanno componenti numerate da 1 a n+1, invece che da 0 a n):

```
p = a(n+1)
for i = n:-1:1
    p = p * x + a(i);
end
```

N.B.: se volessimo calcolare il polinomio in un vettore di punti, è sufficiente sostituire l'operazione nel ciclo con

```
p = p .* x + a(i);
```

Questo algoritmo può essere generalizzato al caso del polinomio interpolante (1), utilizzando la definizione incrementale

(2) della base di Newton.

$p = f[x_0, \ldots, x_n]$

per $i = 1 : n$

$p = p \cdot (x - x_{n-i}) + f[x_0, \ldots, x_{n-i}]$

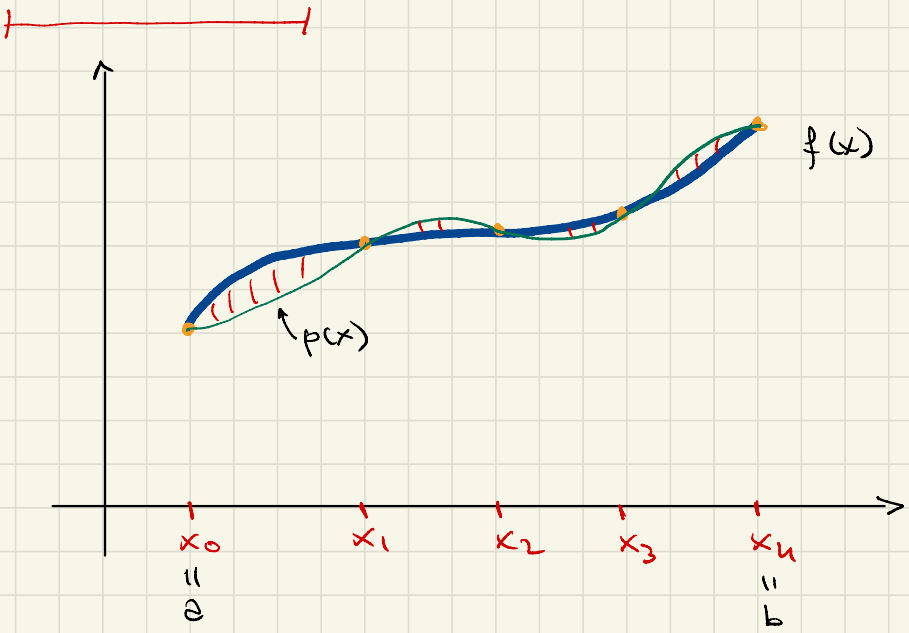
fine per

O, in pseudo-codice Matlab, se x e f sono i vettori con le ascisse e con le differenze divise (vedi (2)), mentre xx è il vettore di punti in cui calcolare il polinomio:

```
p = f(n+1);
for i = n:-1:1
    p = p.*(xx - x(i)) + f(i);
end
```

dove, al solito, il .* è utilizzato per calcolare il polinomio interpolante in un vettore, xx, di ascisse. Pertanto, con un costo di $3n$ flops per ogni punto in

cui viene calcolato il polinomio interpolante.



$p(x)$ è il polinomio interpolante $f(x)$ nelle ascisse assegnate. Se definiamo

$$e(x) = f(x) - p(x), \quad \text{[funzione dell'errore]}$$

da cui, ricordando che $p(x_i) = f(x_i)$, otteniamo che:

$$e(x_i) = 0, \quad i = 0, \dots, n.$$

Cosa succede se $x \notin \{x_0, \dots, x_n\}$?